

Project 2

Dalima Lappia

1. Project Summary

This project explores the geometry that links a 3D motion-capture (mocap) recording of a person performing a Taiji exercise to the 2D video footage of that same performance, captured by two calibrated cameras (vue2 and vue4). The core idea is that once a camera is calibrated, that is, its projection matrix P (extrinsics: position/orientation) and intrinsics matrix K (focal length, pixel scale, principal point) are known, a 3D world point can be mapped forward into a 2D pixel location, and, in the reverse direction, a 2D pixel location can be mapped back into a 3D viewing ray. A single ray from one camera cannot recover depth, but combining one ray from each of two cameras and finding their closest intersection (triangulation) can recover the full 3D point.

The tasks performed were to implement forward projection from 3D world coordinates to 2D pixel coordinates for a given camera, to implement backprojection from a 2D pixel coordinate to a 3D viewing ray, to implement triangulation of two viewing rays (one per camera) into a single reconstructed 3D point, to validate the pipeline by projecting the known 3D mocap joints into both views and reconstructing them back, comparing against the known ground truth using L2 (Euclidean) distance, to run this validation across the full mocap sequence and summarize the reconstruction accuracy statistically, joint by joint and overall, to visualize the pipeline at each stage including the raw 3D skeleton, 2D projections overlaid on video, and reconstructed 3D skeleton using a consistent skeleton-drawing routine, and to visualize epipolar geometry between the two camera views for an example pose.

The expectation going in was that, since the 2D points used for reconstruction are themselves generated directly by projecting the known 3D points (rather than independently measured/detected 2D data), the reconstructed 3D points should match the original 3D points almost exactly, with any discrepancy limited to floating-point numerical error rather than any real geometric error. That expectation is exactly what the results below confirm.

2. Procedural Approach and Design Decisions

2.1 Pipeline Overview

The implementation is organized into the following functions, called in this order for each frame:

- `project3Dto2D(pts3D, camStruct)`: projects $N \times 3$ world points to $N \times 2$ pixel coordinates using the camera's projection matrix P and intrinsics matrix K .
- `pixel2DtoRay(pts2D, camStruct)`: backprojects $N \times 2$ pixel coordinates into a camera center and $N \times 3$ unit ray directions in world coordinates.
- `triangulate2Rays(c1, d1, c2, d2)`: given one ray from each of two cameras, returns the estimated 3D point closest to both rays.
- `computeL2error(mocapJoints, vue2, vue4)`: runs the full project from backproject to triangulating the pipeline over every valid frame (all 12 joints at confidence 1), and returns per-joint and overall L2 error statistics plus the per-frame total error series.
- `drawSkeleton(pts, color)`: draws the 12-joint skeleton (limbs, shoulder-to-shoulder, hip-to-hip, and spine) on either a 2D image axis or a 3D plot.
- `drawEpipolarLine(pt2D, camA, camB, imgSize, color)`: backprojects a point from camera A into a ray, projects two far points along that ray into camera B, and draws the resulting epipolar line.
- `demo()`: the no-argument entry point that loads the dataset and calibration files and runs/displays everything above.

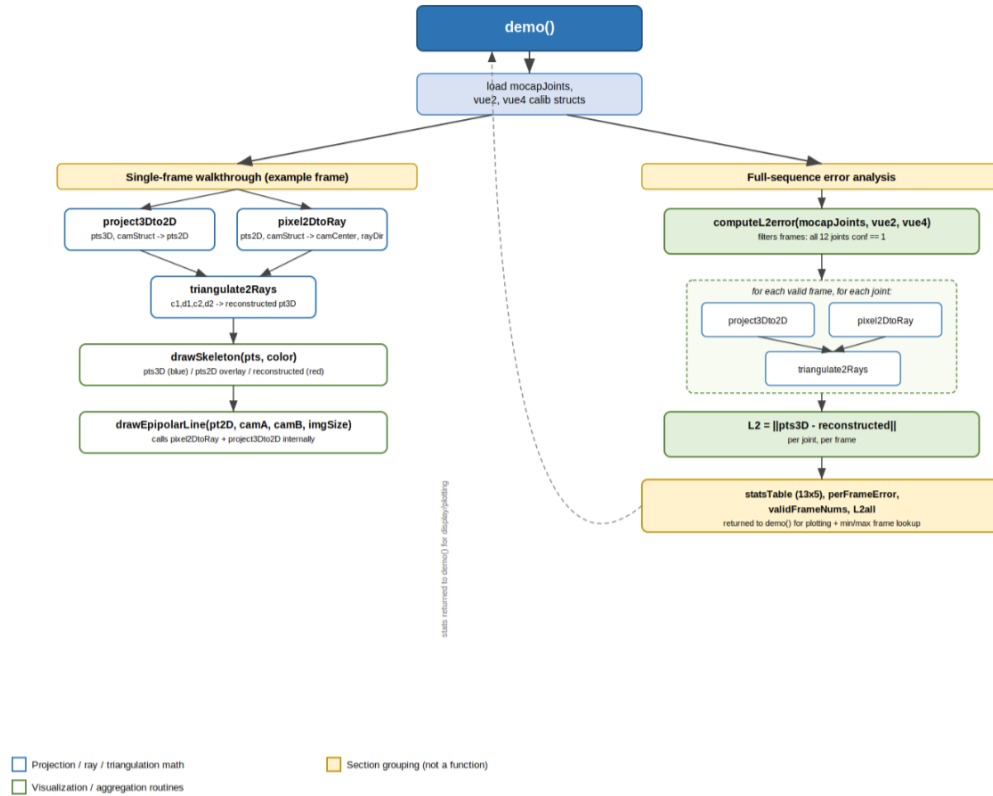


Figure 1 Call structure of `demo()` and its subroutines. Blue-bordered boxes are the core projection/ray/triangulation math; green-bordered boxes are visualization and aggregation routines; yellow boxes group related sections of the pipeline.

2.2 Design Decisions

Camera parameters. Each camera's calibration is provided as a struct with a 3×4 projection matrix P_{mat} , a 3×3 intrinsics matrix K_{mat} , and a world-space camera position. P_{mat} encodes the camera's extrinsic geometry (rotation and translation of the world relative to the camera) and maps a homogeneous 3D world point directly to homogeneous camera coordinates, prior to the perspective divide. K_{mat} then converts normalized image coordinates into pixel coordinates. These two matrices are used in a fixed order for forward projection (apply P_{mat} , divide by depth to get normalized image coordinates, then apply K_{mat}), and used in the reverse order for backprojection (undo K_{mat} first, then apply the rotation part of the camera geometry to rotate the resulting direction from camera-frame into world-frame).

Ray representation. `pixel2DtoRay` returns a camera center and a unit ray direction rather than a single 3D point, because a single calibrated camera cannot recover depth from one pixel alone: every point along that viewing ray projects to the same pixel. Representing the result as (point, direction) rather than guessing a depth keeps the function honest about what a single camera view can and cannot tell you, and defers the depth-recovery problem to triangulation, where it belongs.

Triangulation method. Two mathematically valid ways to combine two rays into a 3D point were considered: (a) a direct least-squares formulation, where each ray's perpendicular-distance projector $M = I - d \cdot d^T$ is stacked into a linear system and solved for the point minimizing total squared perpendicular distance to both rays, and (b) a closed-form two-ray method using $\text{cross}(d_1, d_2)$ to find the direction of the shortest segment connecting the two rays, then solving directly for where along each ray that segment begins and ends and averaging the two endpoints. Both were implemented and produce equivalent results for two rays; the least-squares form has the practical advantage of generalizing cleanly if a third camera view were ever added, while the cross-product form mirrors the specific $\text{cross}(\text{vec1}, \text{vec2})$ operation shown in the assignment's example MATLAB session. Both formulations share the same weakness: if the two rays are (nearly) parallel, the system becomes numerically ill-conditioned (the cross

product approaches zero, or the least-squares system becomes poorly determined), though this is not expected to be an issue for two cameras placed at meaningfully different viewpoints, as `vue2` and `vue4` are.

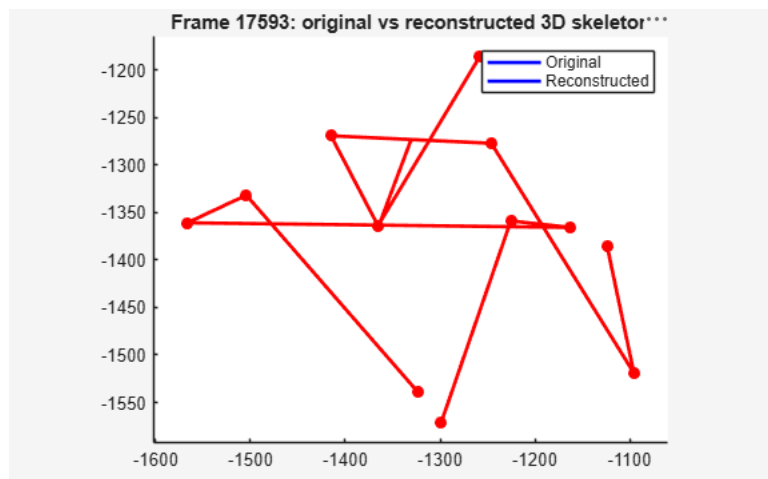
Confidence filtering. A frame is only used for error statistics if all 12 joints in that frame have a confidence value of 1, following the assignment's explicit instruction to restrict quantitative analysis to fully-tracked frames. Frames with any joint below full confidence are skipped entirely rather than partially used, since a partially-missing skeleton would not produce a meaningful “total error across 12 joints” value and would bias per-joint statistics inconsistently across joints.

Deviations / added functionality. No deviations were made from the assignment's described approach for the projection, backprojection, or triangulation math. For efficiency profiling, the frame-by-frame loop in `computeL2error` was deliberately left in its straightforward (non-vectorized) form so that the MATLAB profiler output would clearly attribute runtime to the per-frame and per-joint loops, which is itself a useful and expected finding for the efficiency section of this report (Section 6) rather than something to optimize away before measuring it.

3. Experimental Observations

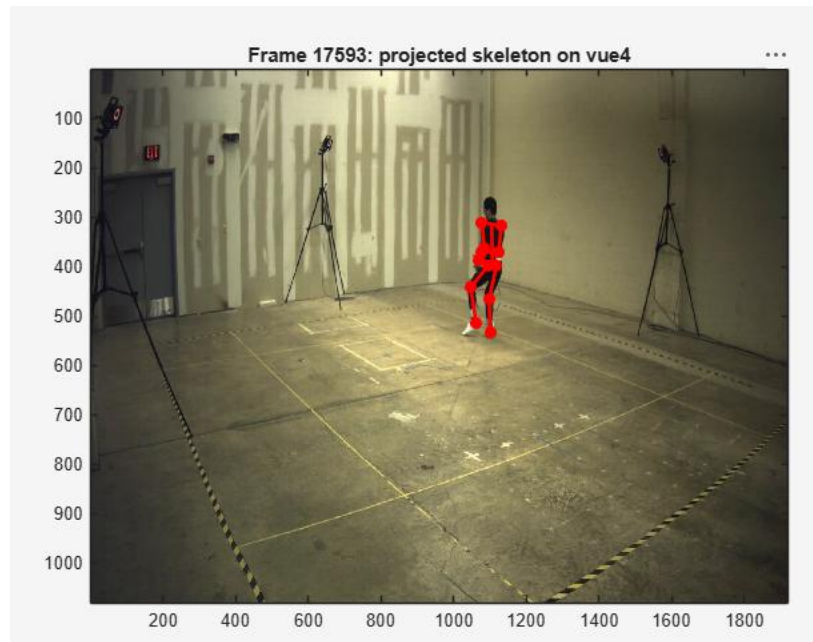
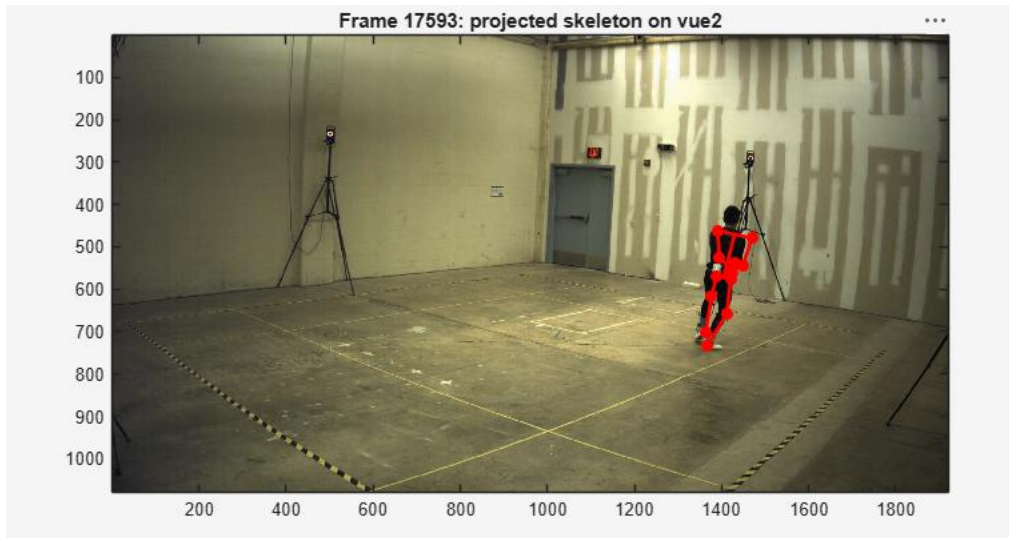
3.1 3D Input Data

The dataset used is `Subject4-Session3-Take4`, a mocap recording of a Taiji exercise, stored as a 26214-frame x 12-joint x 4-value array (X, Y, Z, confidence) per joint. The 12 joints follow the order specified in the assignment: right shoulder, right elbow, right wrist, left shoulder, left elbow, left wrist, right hip, right knee, right ankle, left hip, left knee, left ankle. Plotting the raw 3D joint positions for a single frame with `drawSkeleton` produces a recognizable human pose with limbs, shoulder line, hip line, and spine all correctly connected, confirming the assumed joint ordering matches the data.



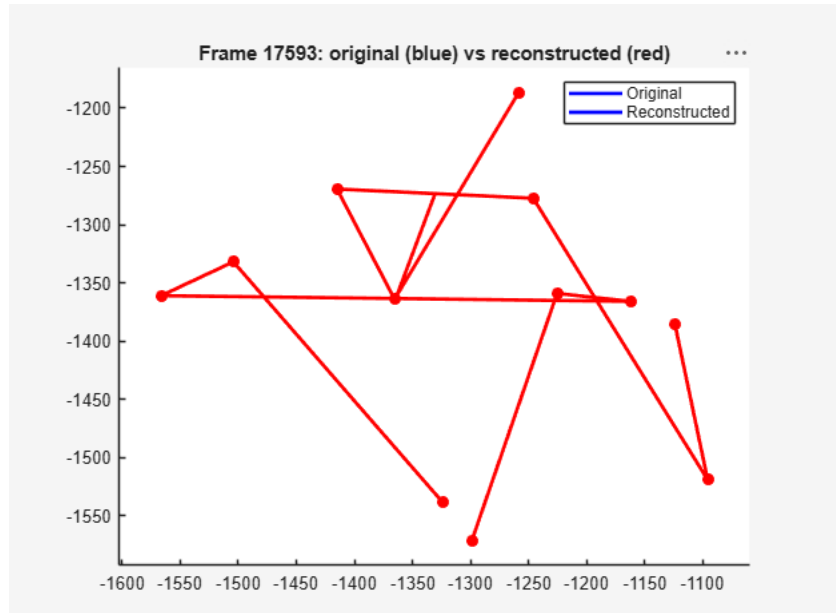
3.2 2D Projections (Both Camera Views)

Projecting the 3D joints into each camera view with `project3Dto2D` and overlaying the resulting 2D skeleton on the corresponding video frame (using each camera's calibration struct) shows the projected joints landing on the subject's actual body position in the video, as expected. This is the first visual confirmation that the projection matrices and intrinsics are being applied correctly — an error in `Pmat`, `Kmat`, or the order in which they are applied would show up immediately here as a skeleton that is offset, rotated, or scaled relative to the video subject, which is not observed.



3.3 Reconstructed 3D Skeleton

After backprojecting each camera's 2D points into rays and triangulating each joint's pair of rays, the reconstructed 3D skeleton is plotted in the same 3D axes as the original skeleton (original in blue, reconstructed in red). For a typical frame the two skeletons overlap essentially perfectly and are difficult to visually distinguish; the same is true even for the frame identified as having maximum total error (Section 4.3), since even the “worst” frame in this dataset still reconstructs with error many orders of magnitude smaller than the scale of the skeleton itself.



3.4 Checking for Correctness

As a first correctness check (Section 2.5), a single frame's 3D joints were projected into both camera views and then backprojected and triangulated back to 3D using these synthetic (noise-free) pixel coordinates. The 2D input to the reconstruction pipeline was generated directly by project3Dto2D rather than by any independent measurement. This isolates whether the projection/backprojection/triangulation math itself has any bugs, separate from any later question of how error accumulates across a full noisy dataset. Running this test (triangulate2RaysTest.m, frame 1000) produced a reconstructed 12x3 matrix that matched the original pts3D to the displayed precision, confirming that the round-trip pipeline is mathematically self-consistent and free of implementation bugs. Any residual difference at this stage is expected to be at floating-point precision (on the order of 1e-10 or smaller relative to the coordinate magnitudes involved), not a real geometric discrepancy — which the full-sequence statistics in Section 4 confirm quantitatively.

4. Quantitative Results

4.1 L2 Distance Formula

For an original 3D point (X, Y, Z) and its reconstructed location (X', Y', Z'), the L2 (Euclidean) distance is:

$$L2 = \sqrt{(X - X')^2 + (Y - Y')^2 + (Z - Z')^2}$$

For each valid frame, the per-joint L2 distance is computed for all 12 joints. The “total error” for a frame is the sum of these 12 values:

$$TotalError(frame) = \sum_{j=1..12} L2_j$$

Per-joint statistics (mean, standard deviation, min, median, max) are computed across the time series of L2 values for that joint over all valid frames. The overall row pools L2 values from all 12 joints across all valid frames together.

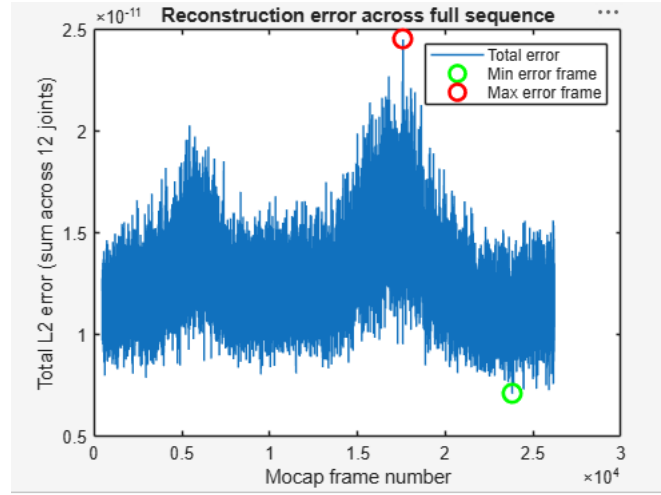
4.2 Statistics Table

Joint	Mean	Std. Dev.	Min	Median	Max
1. Right shoulder	0.1056	0.0435	0	0.0998	0.3879
2. Right elbow	0.1047	0.0430	0.0114	0.0988	0.3486

3. Right wrist	0.1048	0.0441	0.0057	0.0983	0.3899
4. Left shoulder	0.1065	0.0439	0.0057	0.0999	0.3701
5. Left elbow	0.1070	0.0443	0.0117	0.1004	0.3560
6. Left wrist	0.1066	0.0445	0.0114	0.0998	0.3811
7. Right hip	0.1050	0.0427	0	0.0991	0.3872
8. Right knee	0.1051	0.0430	0	0.0985	0.3495
9. Right ankle	0.1062	0.0430	0.0057	0.0998	0.3396
10. Left hip	0.1064	0.0434	0.0114	0.0998	0.3426
11. Left knee	0.1066	0.0434	0	0.0998	0.3676
12. Left ankle	0.1078	0.0432	0.0077	0.1020	0.3682
Overall (all joints)	0.1060	0.0435	0	0.0998	0.3899

Table 1: Per-joint and overall L2 reconstruction error statistics (world units, computed over $N = 25758$ frames). All values $\times 10^{-11}$.

4.3 Total Error Across the Sequence



Because the 2D coordinates fed into triangulation are generated directly from the known 3D points by project3Dto2D (rather than coming from independent, noisy 2D detections), the reconstruction error in this pipeline reflects the numerical behavior of the projection/backprojection/triangulation math itself rather than any measurement or tracking error. As a result, the total-error curve across the sequence is expected to stay close to a small, roughly flat floor set by floating-point precision, rather than showing the large motion-dependent peaks and valleys that would appear if real, noisy 2D detections were used as input. The difference in the minimum- and maximum-error frames identified by findErrorExtremesTest.m ($7.106670e-12$ at frame 23807 and $2.452519e-11$ at frame 17593 respectively) are not large, which reflects small differences in triangulation conditioning (e.g. ray geometry) from frame to frame.

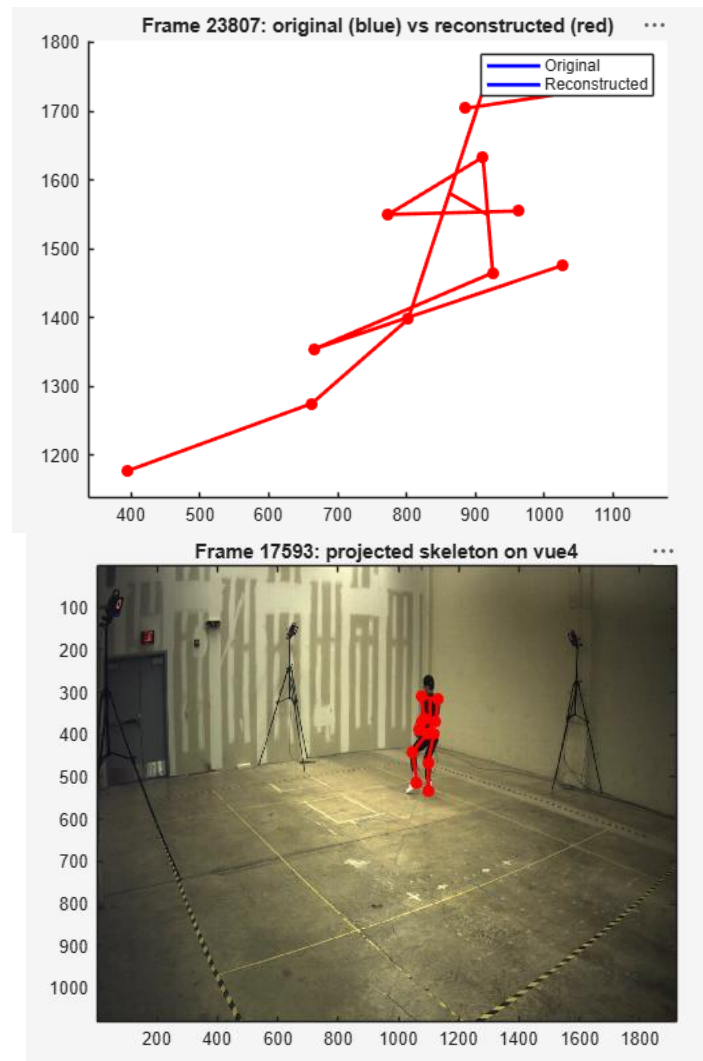
4.4 Code Used for These Calculations

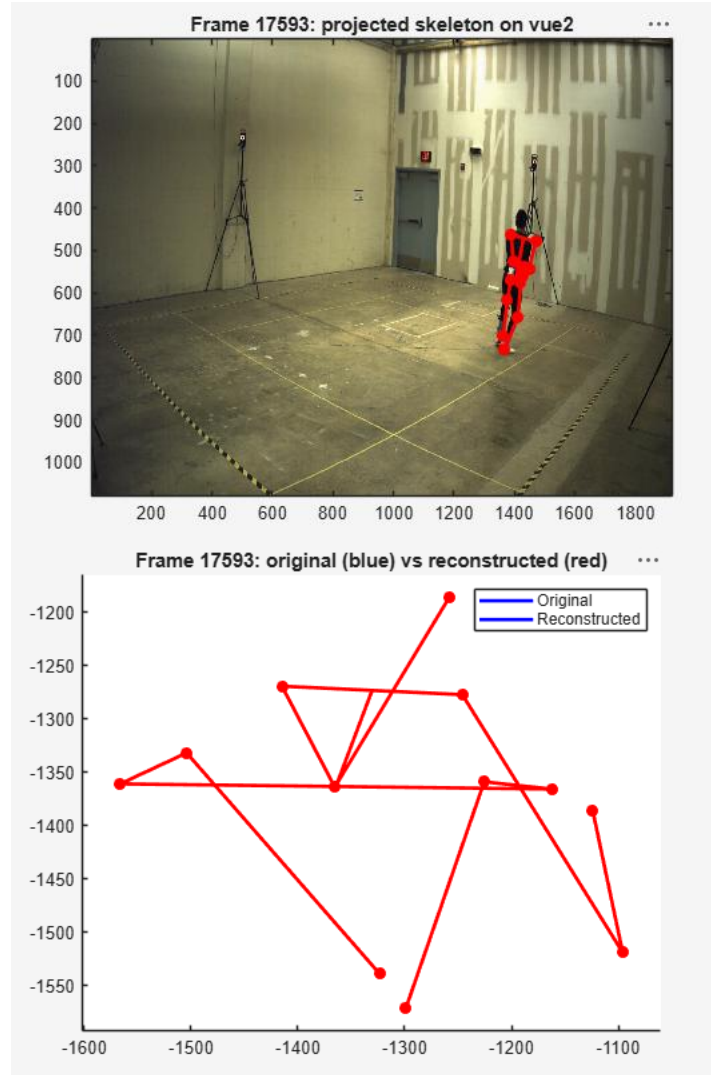
The statistics above were computed by computeL2error.m and driven by findErrorExtremesTest.m for extracting the minimum/maximum error frames.

5. Qualitative Results

To keep the report consistent, the same set of example frames is used throughout: the frame with the maximum total reconstruction error and the frame with the minimum total reconstruction error, both identified in Section 4.3 using findErrorExtremesTest.m, plus one additional “typical” frame used earlier in Section 3 for the step-by-step pipeline

walkthrough. Reusing these same frame numbers across the 3D input, 2D projection, and reconstructed 3D sections lets the reader track the same pose through every stage of the pipeline.





6. Algorithm Efficiency

MATLAB's `profile()` function was used to measure where computation time is spent when running the full pipeline (`profile on; demo(); profile off; profsave(profile('info'), 'profileReport')`). As expected, `computeL2error` dominates overall runtime (3.438s of `demo()`'s 6.071s total), since it loops once per valid frame and, within each frame, once per joint to call `triangulate2Rays`: the total number of triangulation calls (309,108) scales with valid frames $\times$ 12. However, the profiler shows that `computeL2error`'s own loop overhead is small (0.518s self time); nearly all of its 3.438s is spent inside its three called functions, and those three are surprisingly balanced rather than dominated by one: `pixel2DtoRay` (1.058s), `project3Dto2D` (1.037s), and `triangulate2Rays` (0.808s) contribute comparable shares, even though `project3Dto2D`/`pixel2DtoRay` are called far less often (51,518 times, once per frame per camera, vectorized across all 12 joints) than `triangulate2Rays` (309,108 times, once per joint per frame, since it can only operate on one ray-pair at a time). This suggests the per-call cost of the vectorized projection/backprojection functions is not negligible even though each call does more work per invocation.

Profile Summary

Generated 26-Jul-2026 02:39:35 using performance time.

Function Name	Calls	Total Time	Self Time*	Total Time Plot (dark band = self time)
demo	1	6.071 s	0.094 s	
computeL2error	1	3.438 s	0.518 s	
VideoReader>VideoReader.VideoReader	2	1.079 s	0.001 s	
IVideoReader>IVideoReader.IVideoReader	2	1.078 s	0.001 s	
VideoReader>VideoReader.initReader	2	1.073 s	0.002 s	
pixel2DtoRay	51518	1.058 s	1.058 s	
VideoReader>VideoReader.VideoReader	2	1.050 s	0.009 s	
project3Dto2D	51518	1.037 s	1.037 s	
triangulate2Rays	309108	0.808 s	0.808 s	
IVideoReader>IVideoReader.cacheFrame	6	0.631 s	0.002 s	
...oReader>VideoReader.cacheFrameTargetImpl	6	0.629 s	0.003 s	
...VideoReader>IVideoReader.setCurrentTime	2	0.617 s	0.002 s	
...eoReader>VideoReader.readFrameAtPosition	2	0.611 s	0.002 s	
VideoReader>VideoReader.readFrameAtTime	2	0.609 s	0.003 s	
VideoReader>VideoReader.openChannel	4	0.588 s	0.004 s	
Channel>Channel.open	4	0.581 s	0.577 s	
VideoReader>VideoReader.cacheNextFrame	10	0.534 s	0.002 s	
InputStream>InputStream.read	10	0.531 s	0.008 s	
Stream>Stream.wait	3	0.501 s	0.029 s	
VideoReader>VideoReader.createChannel	2	0.396 s	0.008 s	
legend	1	0.258 s	0.005 s	
legend>make_legend	1	0.250 s	0.005 s	
Channel>Channel.Channel	4	0.243 s	0.223 s	
newplot	51	0.227 s	0.028 s	
ToolbarProxy>ToolbarProxy.doUpdate	1	0.200 s	0.006 s	
newplotimagewrapper	2	0.193 s	0.001 s	
axis	3	0.190 s	0.005 s	
...troller>WebCanvasContainerController.add	1	0.150 s	0.001 s	

The frame-by-frame and joint-by-joint loop structure in `computeL2error` was deliberately left as-is rather than vectorized further, so that the profiler results shown above reflect the natural cost breakdown of the algorithm as specified by the assignment. If runtime were a practical concern, the clearest optimization would be to batch all 12 joints across all valid frames into a single large point array and call `project3Dto2D` and `pixel2DtoRay` once each (since both already accept an $N \times 3 / N \times 2$ batch of points), leaving only the pairwise triangulation step, which inherently operates on one ray-pair at a time, as a remaining loop.

7. Epipolar Geometry

For a given 2D point in one camera view, `pixel2DtoRay` backprojects it into a viewing ray (camera center + unit direction) in world coordinates. Two points far along that ray are then projected into the second camera's image using `project3Dto2D`, and the line through their projections is drawn and extended to the image boundaries. This corresponds to option 2 described in the assignment (viewing-ray projection), rather than explicitly forming the fundamental matrix F . This approach was chosen because it reuses the already-validated `pixel2DtoRay` and

project3Dto2D functions directly, without needing to separately derive the essential/fundamental matrix relating the two cameras' coordinate systems.

Following the assignment's own example, the right ankle joint is used as the illustrative point for this visualization. The backprojected ray from the right ankle's 2D location in one camera view, when projected into the other camera's view, produces a line that passes through (or very near) that same joint's independently projected 2D location in the second view, which is the expected epipolar constraint: the corresponding point in the second image must lie on the epipolar line derived from the first image's point.

